Helvetica Now Display
Step 1: Defining the Constraints

Functional Requirements

- Crawl from seed URLs
- Extract text data for LLM training
- Store data persistently

Non-Functional Requirements

- **Fault Tolerance:** Resume without data loss
- **Politeness:** Respect robots.txt & server load
- **Efficiency:** No duplicate processing
- **Scale:** 10 Billion pages

Goal: Raw text extraction for LLM training (not search indexing).

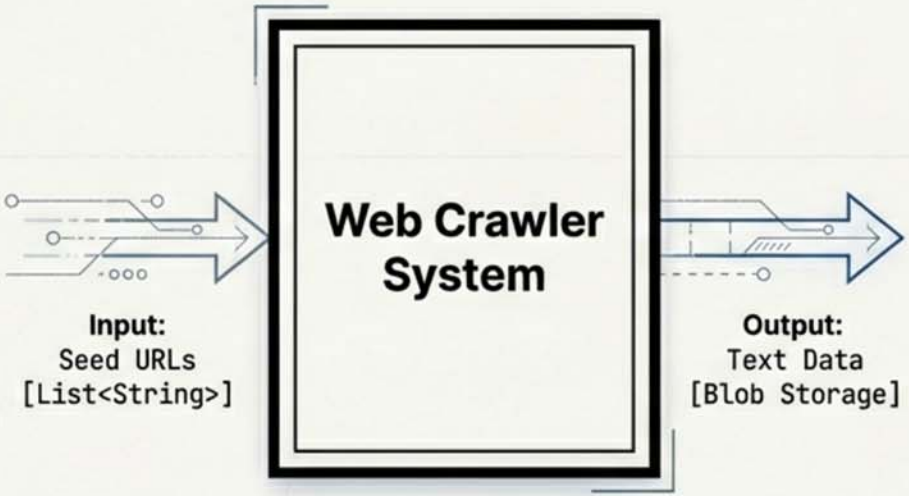
The Constraint: The crawl must be completed in exactly **5 DAYS.**

SLATE GREY

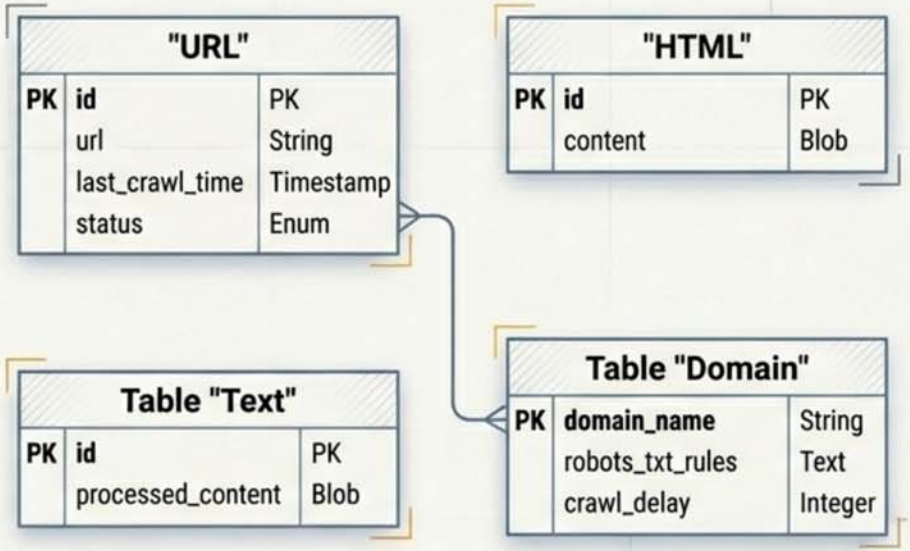
PROCESS

Steps 2 & 3: The Entities and Interface

The Interface 40%

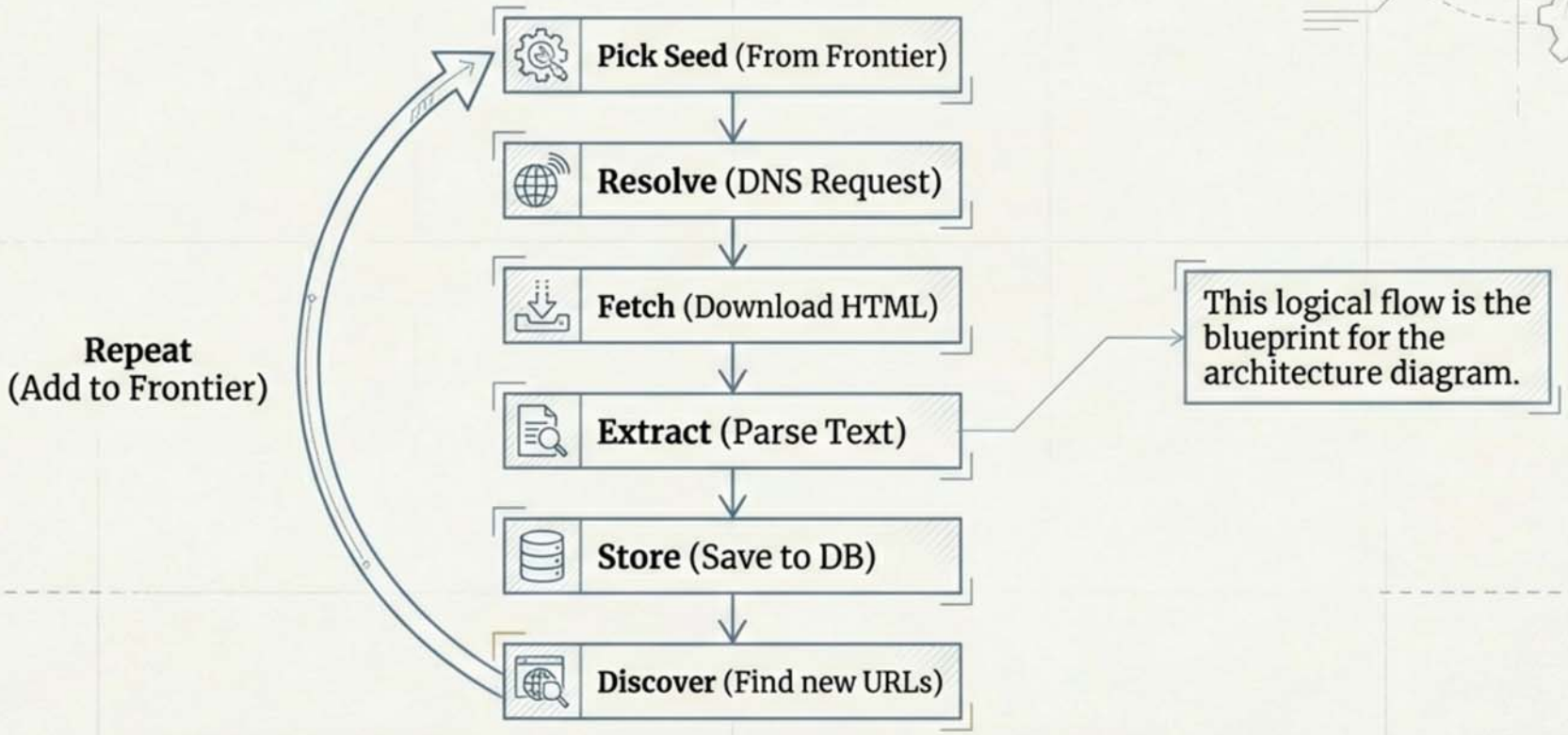


Core Entities 60%

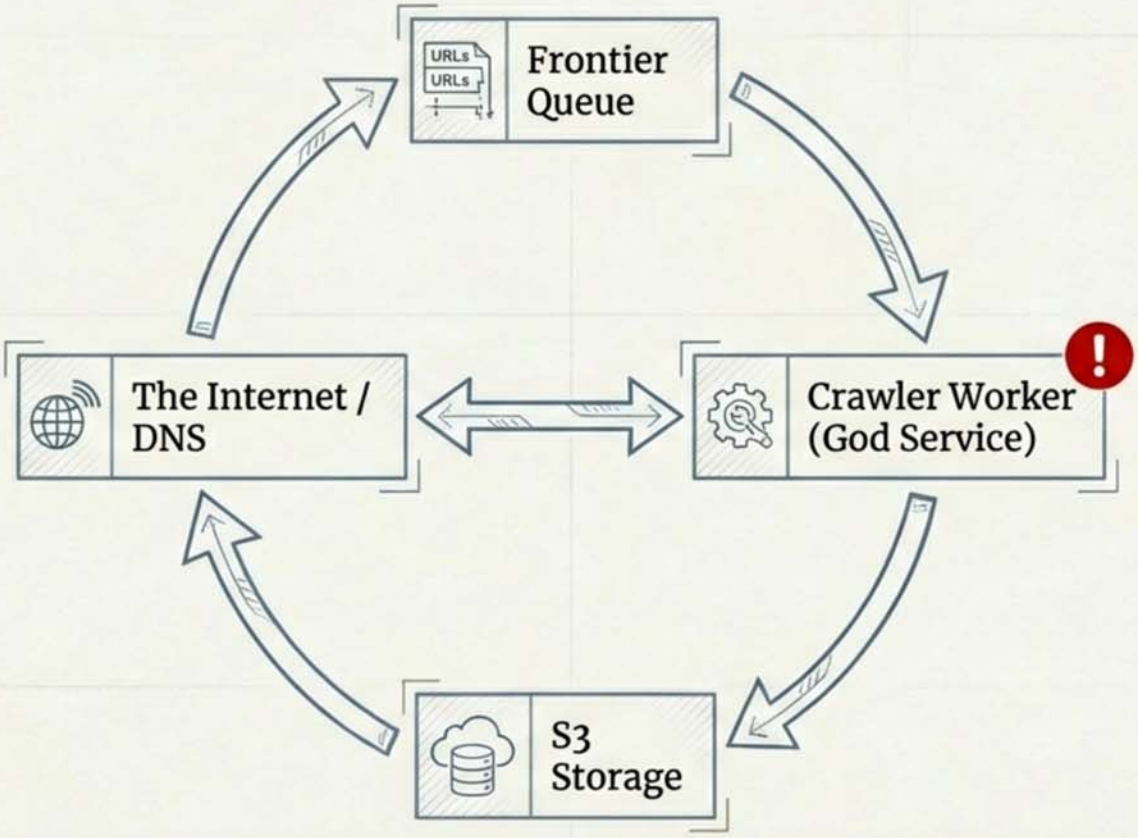


Step 4: Tracing the Data Flow

The lifecycle of a single URL

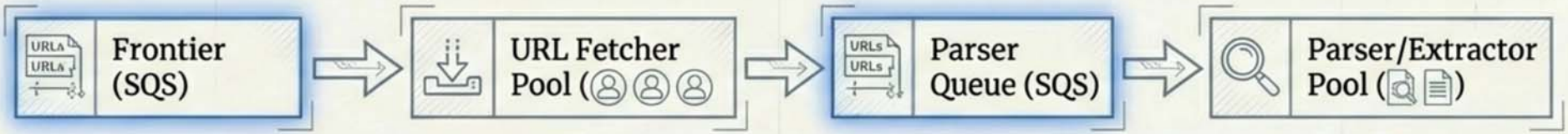


Step 5: High-Level Design (The Naive Solution)

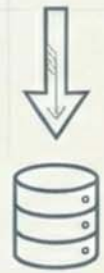


CRITICAL FLAW: Coupling Fetching (IO Bound) with Parsing (CPU Bound). One crash kills the whole pipeline.

Deep Dive 1: Fault Tolerance via Pipelining



S3 (Raw HTML)



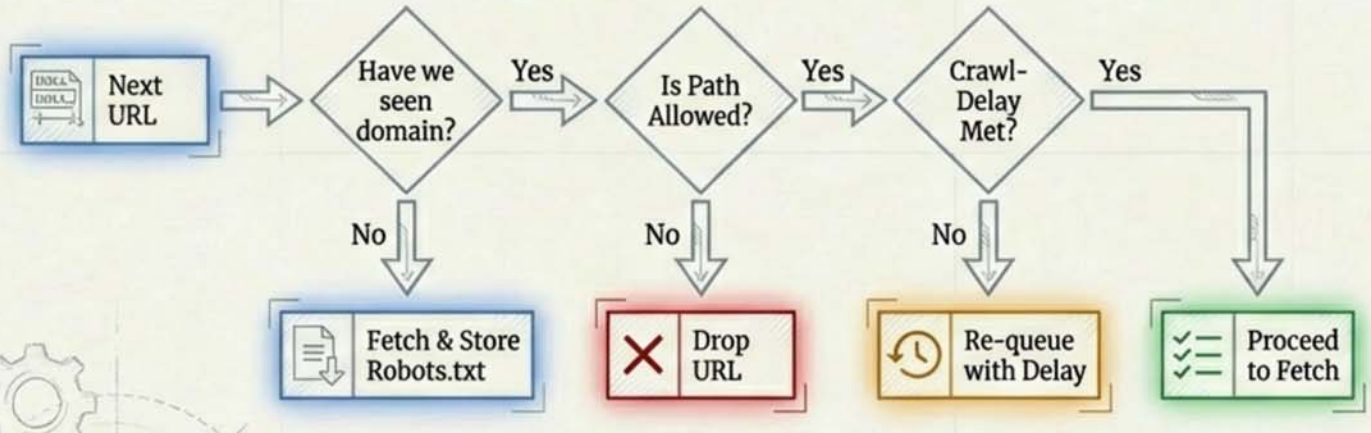
S3 (Text Data)

Why SQS over Kafka?

- ✓ **SQS:** Visibility Timeout = Automatic Retries (Good for discrete tasks)
- ✗ **Kafka:** Manual Offset Management (Complex for retry logic)

Deep Dive 2: Politeness & Robots.txt

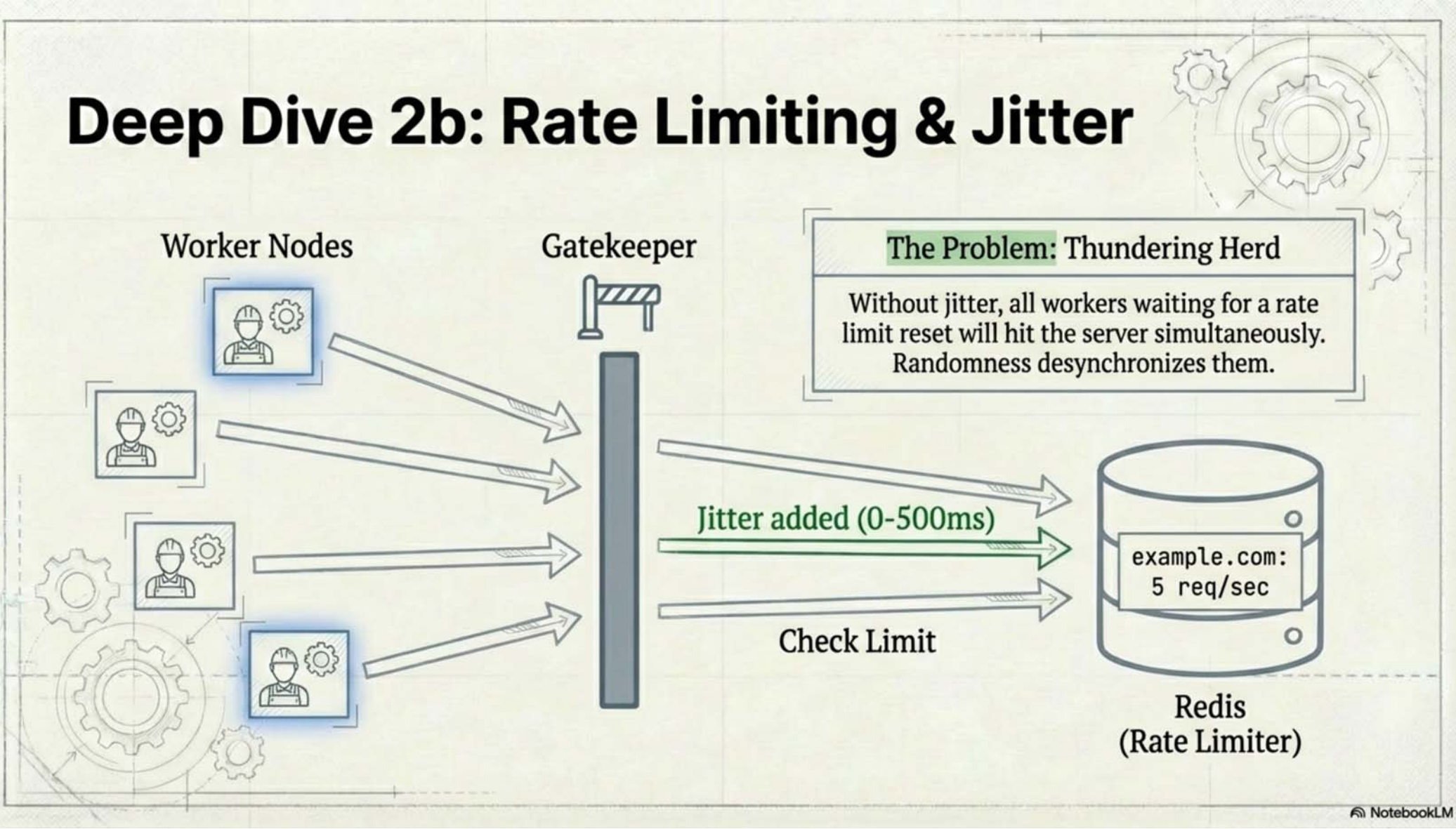
Politeness Check Logic



Domain Metadata

domain_hash	rules_json	last_request_timestamp
0xabc123	{ "User-agent": "*", "Disallow": "/admin/" }	1678886400
0xabc123	{ "User-agent": "*", "Disallow": "/admin/" }	1678886400
0xabc123	…	1678886400
0xabc123	{ "User-agent": "*", "Disallow": "/admin/" }	1678886400
0xabc123	{ "User-agent": "*", "Disallow": "/admin/" }	1678886400

Deep Dive 2b: Rate Limiting & Jitter



Deep Dive 3: Scale & Estimation

Target: **10 Billion Pages / 5 Days**

Throughput Needed: **~23,000 pages/sec**

Bandwidth Constraints:

- AWS Network Optimized Instance: **~400 Gbps**
- Average Page Size: **2MB**
- Theoretical Max: **25k pages/sec/machine**

Real World Adjustment:

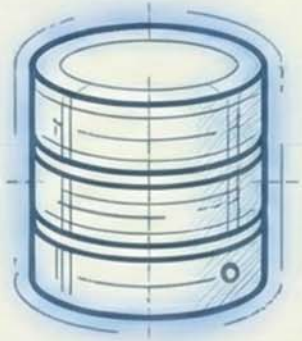
- Efficiency Factor: 30% (Latency, DNS, Errors)
- **Effective Throughput: ~7,500 pages/sec/machine**

Result: 23,000 / 7,500 ≈ 4 Machines needed for Fetching.

Deep Dive 4: Efficiency & Deduplication

URL Deduplication

Check Index:
Have we seen
this URL string?



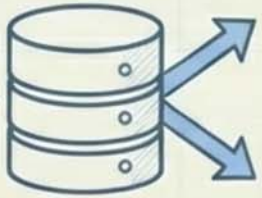
URL Metadata

Content Deduplication

HTML

SHA-256

Hash
String



Global Index
(DB)

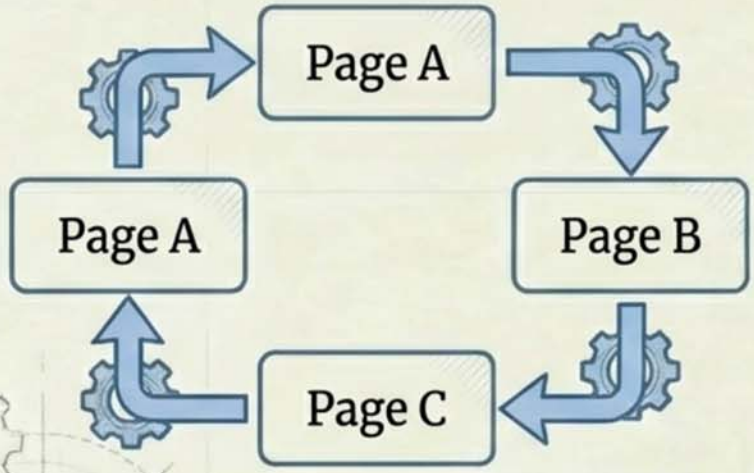
Hash Exists
-> Discard

Hash New
-> Store

Trade-off: Bloom Filters (Space efficient, False Positives) vs. DB Index (Reliable, Disk usage). For 10B pages, DB Index is safer.

Deep Dive 5: Traps & Edge Cases

Crawler Trap



Website structure shows Page A -> Page B -> Page C -> Page A (infinite loop).

The Solution

Max Depth Logic

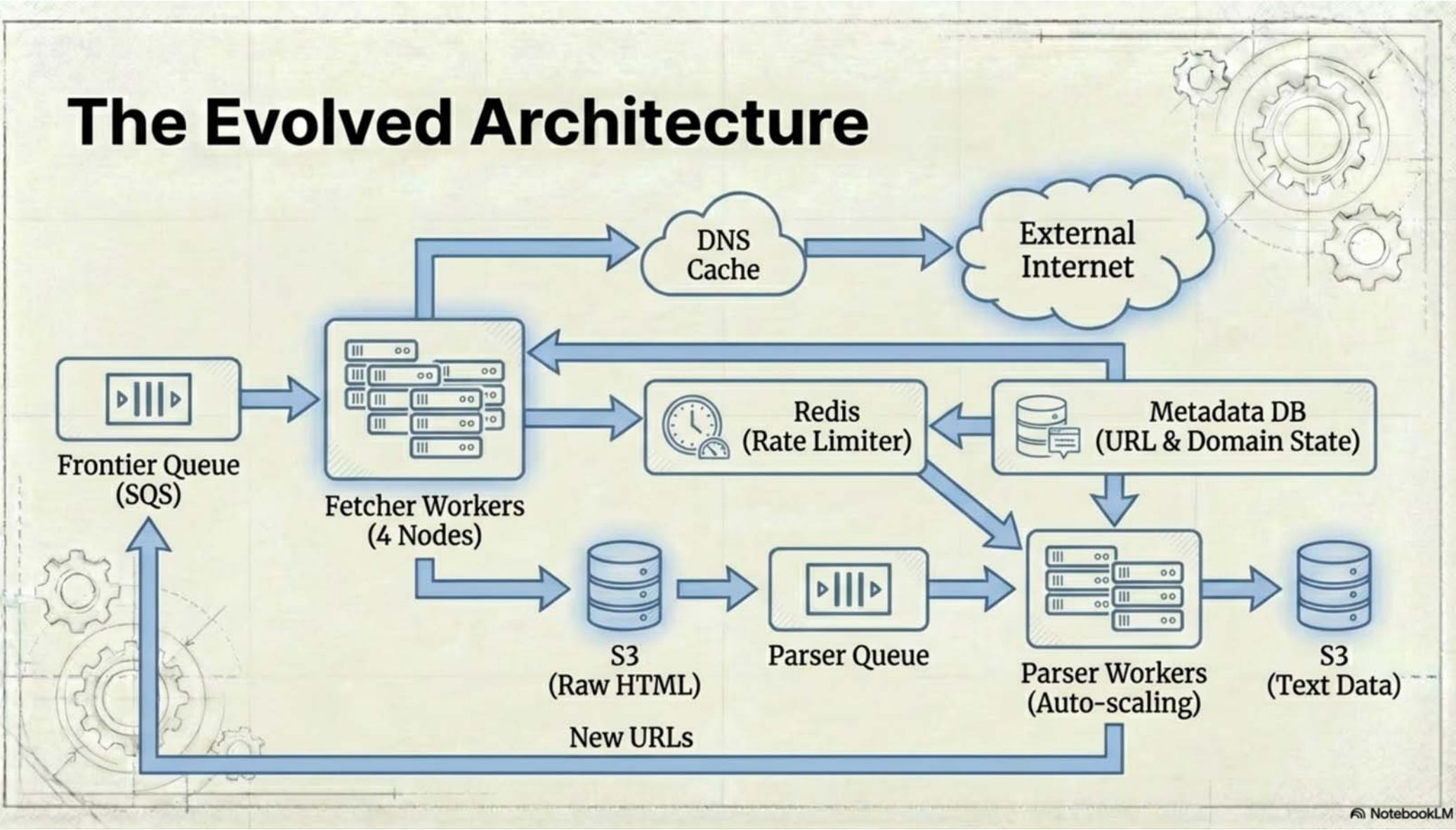
```
if (url.depth > MAX_DEPTH) {
  stop_crawling();
}
```

Dynamic Content (React/Angular)

Slow Lane

Headless Browser (Puppeteer). CPU Intensive. Segregate to specific worker pool.

The Evolved Architecture



Summary & Checklist

- **Pipelining** = Fault Tolerance (Split Fetch/Parse)
- **Redis + Jitter** = Politeness & Thundering Herd Protection
- **Content Hashing** = Efficiency (Deduplication)
- **SQS Visibility Timeout** = Simple Retry Logic

A web crawler is a system of constraints. Success isn't just about speed; it's about balancing speed with politeness and robustness.